

Software Requirements Specification

SRS · Version 1.0 · Confidential

Document Type	Software Requirements Specification
Project	FinNewt.bi — Personal Finance Intelligence
Version	1.0
Classification	Confidential
Prepared By	Engineering Lead — 30+ Years SDE
Date	2025

Table of Contents

Section	Title	Page
1	Introduction	3
2	System Overview & Architecture	4
3	Functional Requirements	5
3.1	Authentication Module	5
3.2	Dashboard Module	6
3.3	Transactions Module	6
3.4	Budget Module	7
3.5	Analytics Module	7
3.6	Goals Module	8
3.7	Notifications Module	8
3.8	Reports Module	8
4	Non-Functional Requirements	9
5	Database Design	10
6	API Design	11
7	Security Design	12
8	Insights Engine Logic	13
9	UI/UX Requirements	14
10	Future Enhancements	14

1. Introduction

1.1 Purpose

FinNewt.bi is a modern fintech-based personal finance intelligence platform designed to empower individuals to achieve financial clarity and discipline through data-driven insights. The platform addresses a critical gap in the personal finance space — the absence of an intelligent, unified system that not only tracks money but actively analyzes behavioral patterns and drives positive financial habits.

◆ Core Capabilities

- Track income and expenses with granular categorization
- Analyze spending behavior using rule-based intelligence engine
- Manage category-based budgets with real-time alert thresholds
- Monitor progress toward defined savings goals with projections
- Receive contextual smart financial insights and recommendations
- Visualize financial trends through interactive multi-dimensional charts

1.2 Scope

The application delivers a comprehensive financial management ecosystem targeting web and mobile-responsive platforms. The scope encompasses secure authentication flows, full transaction lifecycle management, dynamic analytics dashboards, and an automated notifications infrastructure.

◆ Platform Coverage

Platform	Support Level	Description
Web Application	Primary	Full feature set, desktop-optimized layout
Tablet (Responsive)	Secondary	Adapted grid layout, touch-friendly controls
Mobile Browser	Tertiary	Condensed navigation, essential features

1.3 Objectives

◆ Primary Objectives

- Simplify personal finance tracking through an intuitive interface
- Improve user financial awareness via behavioral pattern analysis
- Encourage consistent savings discipline through goal accountability
- Detect and surface overspending patterns proactively

◆ Technical Objectives

- Build a horizontally scalable fintech microservice architecture
- Implement robust inter-service communication with retry semantics

- Leverage Redis caching for sub-500ms API response times
- Deliver enterprise-grade dashboard UI with accessibility compliance

1.4 Target Users

User Type	Profile	Key Use Cases
Standard User	Individuals managing personal finances	Expense logging, budget tracking, savings goals
Advanced User	Multi-goal & analytics-focused users	Trend analysis, subscription tracking, custom reports
Admin (Future)	Application operations team	Platform monitoring, analytics management, user support

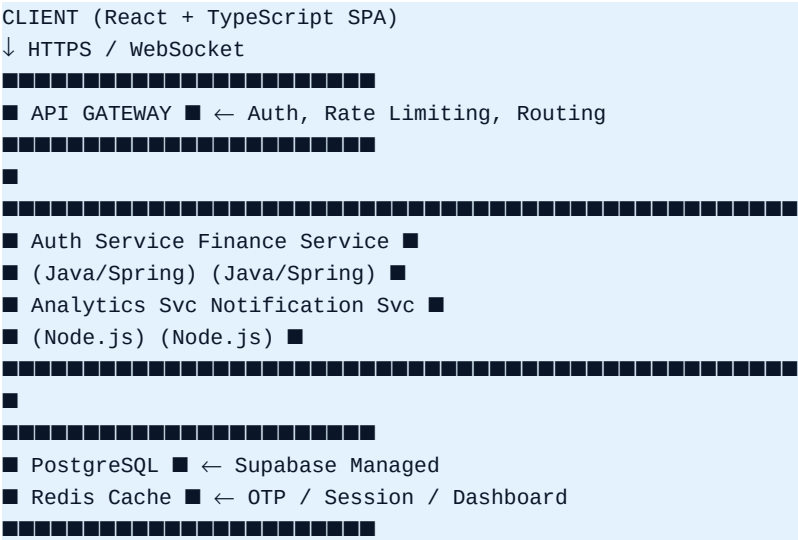
1.5 Technology Stack

Layer	Technology	Rationale
Frontend	React + TypeScript + TailwindCSS	Type-safe, component-driven, rapid styling
Charting	Recharts + Framer Motion	Declarative SVG charts, fluid animations
Backend Core	Java Spring Boot	Enterprise-grade REST APIs, mature ecosystem
Backend Services	Node.js Microservices	Event-driven analytics & notification pipelines
Database	Supabase PostgreSQL	Managed RDBMS, realtime subscriptions, RLS
Cache	Redis	OTP storage, rate limiting, dashboard query cache
Auth	JWT + SMTP OTP	Stateless sessions, email-verified registration
DevOps	Docker + GitHub Actions	Containerized deployments, automated CI/CD
Hosting	Render / Railway / AWS	Scalable cloud deployment options

2. System Overview & Architecture

2.1 High-Level Architecture

FinNewt.bi follows a microservice-oriented architecture with a central API Gateway mediating all client-server interactions. Each service owns its domain logic independently, enabling isolated deployment, scaling, and fault tolerance.



2.2 System Modules

Module	Primary Technology	Description
Authentication	Java Spring Boot	Registration, OTP verification, JWT issuance
Dashboard	React + Spring Boot	Aggregated financial overview and insights panel
Transactions	Java Spring Boot	Full CRUD for income and expense records
Budgets	Java Spring Boot	Category budget limits and overspend detection
Analytics	Node.js	Rule-based insight engine, trend calculation
Goals	Java Spring Boot	Savings goal creation and progress tracking
Notifications	Node.js + SMTP	Automated alerts, reminders, weekly digests
Reports	Node.js + ReportLab	CSV and PDF financial report generation
Settings	React + Spring Boot	User profile management and preferences

3. Functional Requirements

3.1 Authentication Module

The authentication subsystem handles the complete identity lifecycle including registration, OTP-based email verification, secure login, and password reset. All sessions are managed via stateless JWT tokens.

◆ Functional Requirements

ID	Requirement	Priority
FR-A1	User can self-register with name, email, and password	Critical
FR-A2	System sends OTP to registered email via SMTP on registration	Critical
FR-A3	User email must be verified via OTP before account activation	Critical
FR-A4	Passwords must be hashed using BCrypt prior to storage	Critical
FR-A5	Successful login generates a signed JWT with configurable expiry	Critical
FR-A6	OTP must expire after a configurable time window (default: 10 min)	High
FR-A7	Failed OTP attempts must be rate-limited (max 3 attempts)	High
FR-A8	Forgot password flow resets via new email OTP verification	High
FR-A9	Refresh token mechanism to extend active sessions	Medium

3.2 Dashboard Module

The dashboard presents an at-a-glance financial summary, aggregating real-time data across all modules into a unified, visually rich interface.

ID	Requirement	Priority
FR-D1	Dashboard loads complete user financial summary on authentication	Critical
FR-D2	Dashboard displays interactive charts: balance, income, expense trends	Critical
FR-D3	Rule-based insights are generated and displayed dynamically per session	High
FR-D4	Last 5–10 transactions are rendered in the recent activity feed	High
FR-D5	Dashboard data refreshes in real-time or on user-triggered reload	Medium
FR-D6	Monthly income vs. expense comparison KPI cards displayed	High

3.3 Transactions Module

◆ Transaction Data Model

Field	Data Type	Constraints	Description
id	UUID	PK, auto-generated	Unique transaction identifier
user_id	UUID	FK → users.id	Owning user reference
amount	DECIMAL(12,2)	NOT NULL, > 0	Transaction monetary value
category	VARCHAR(50)	NOT NULL, ENUM	Spending/income category
type	ENUM	income expense	Transaction direction
date	DATE	NOT NULL	Transaction date
notes	TEXT	NULLABLE	Optional user annotation
is_recurring	BOOLEAN	DEFAULT false	Recurring transaction flag
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp

◆ Functional Requirements

ID	Requirement	Priority
FR-T1	Authenticated user can create a new income or expense transaction	Critical
FR-T2	User can update any field of an owned transaction record	Critical
FR-T3	User can delete an owned transaction with confirmation prompt	Critical
FR-T4	Transactions list supports filtering by: date range, category, type	High
FR-T5	Transactions are automatically categorized by system-defined rules	High
FR-T6	Recurring transactions auto-populate on their scheduled frequency	Medium
FR-T7	Pagination or infinite scroll implemented for large transaction sets	Medium

3.4 Budget Module

The budget module enables users to set monthly spending limits per category and receive real-time alerts as thresholds are approached or exceeded.

ID	Requirement	Priority
FR-B1	User can create a monthly budget limit for any transaction category	Critical
FR-B2	System tracks cumulative spending per category against defined budgets	Critical
FR-B3	Alert notification triggered when category spending exceeds limit	Critical
FR-B4	Dashboard displays remaining budget per category as progress bars	High
FR-B5	Warning alert triggered at 80% budget utilization threshold	High
FR-B6	Historical budget compliance reports available per month	Medium

3.5 Analytics Module

The analytics engine processes transactional data to produce behavioral insights, spending pattern visualizations, and trend projections using a configurable rule set.

ID	Requirement	Priority
FR-AN1	Analytics data loads dynamically from aggregated transaction history	Critical
FR-AN2	Weekly and monthly spending trends are automatically computed	Critical
FR-AN3	Pie charts showing expense breakdown by category are rendered	High
FR-AN4	Line graphs displaying income vs. expense over time are available	High
FR-AN5	Spending heatmap identifies days/periods of high financial activity	Medium
FR-AN6	Rule-based behavioral insights are generated from spending patterns	High

3.6 Goals Module

ID	Requirement	Priority
FR-G1	User can define a savings goal with a target amount and deadline	Critical
FR-G2	User can manually or automatically allocate amounts toward a goal	Critical
FR-G3	System calculates and displays goal completion percentage in real-time	High
FR-G4	Estimated completion date projected based on current savings velocity	High
FR-G5	Reminder notifications sent when goal milestones are reached	Medium

3.7 Notifications Module

ID	Requirement	Priority
FR-N1	Budget overspend alerts generated and delivered automatically	Critical
FR-N2	All notifications persisted in the database for in-app inbox access	High
FR-N3	Weekly financial summary emails dispatched via scheduled SMTP job	High
FR-N4	Upcoming recurring payment reminders sent 24 hours in advance	Medium
FR-N5	User can configure notification preferences and opt-out per type	Medium

3.8 Reports Module

ID	Requirement	Priority
FR-R1	User can export transaction history as a structured CSV file	High
FR-R2	System generates branded PDF financial reports on-demand	High
FR-R3	PDF reports include embedded charts and visual spending breakdowns	Medium
FR-R4	Reports are filterable by custom date range before generation	Medium

4. Non-Functional Requirements

Category	Requirement	Target / Metric
Performance	Dashboard initial load time	< 2 seconds (P95)
Performance	API response time for standard queries	< 500ms (P95)
Performance	Redis cache hit rate for dashboard queries	> 80%
Security	Password storage mechanism	BCrypt hashing, cost factor ≥ 12
Security	Session management	Signed JWT, configurable expiry, refresh tokens
Security	Transport security	TLS 1.2+ enforced, HSTS headers
Security	OTP validity window	10-minute expiry, 3 max attempts
Security	Rate limiting	100 req/min per IP on auth endpoints
Security	Input handling	Parameterized queries, XSS sanitization
Scalability	Service architecture	Stateless microservices, horizontal scaling
Scalability	Session state	Externalized to Redis, no server affinity
Reliability	Data backup frequency	Daily automated backups, 30-day retention
Reliability	Error handling	Structured logging, retry with exponential backoff
Availability	Platform uptime SLA target	99.0% monthly uptime
Maintainability	Code coverage target	$\geq 80\%$ unit test coverage
Accessibility	Web content accessibility	WCAG 2.1 Level AA compliance

5. Database Design

All persistent data is stored in a managed PostgreSQL instance on Supabase. The schema follows third normal form (3NF) to minimise redundancy. Row-Level Security (RLS) policies ensure strict data isolation between users.

◆ Table: users

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full display name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login and notification email
phone	VARCHAR(20)	NULLABLE	Optional contact number
password_hash	VARCHAR(255)	NOT NULL	BCrypt hashed credential
is_verified	BOOLEAN	DEFAULT FALSE	Email OTP verification status
created_at	TIMESTAMPTZ	DEFAULT NOW()	Account creation timestamp

◆ Table: transactions

Column	Type	Constraints	Description
id	UUID	PK	Unique transaction identifier
user_id	UUID	FK → users.id, NOT NULL	Owning user reference
amount	DECIMAL(12,2)	NOT NULL, CHECK > 0	Monetary value
category	VARCHAR(50)	NOT NULL	Spending/income category
type	VARCHAR(10)	CHECK IN ('income','expense')	Transaction direction
date	DATE	NOT NULL	Transaction date
notes	TEXT	NULLABLE	Optional user note
is_recurring	BOOLEAN	DEFAULT FALSE	Recurring payment flag
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation time

◆ Table: budgets

Column	Type	Constraints	Description
id	UUID	PK	Unique budget identifier
user_id	UUID	FK → users.id	Owning user reference
category	VARCHAR(50)	NOT NULL	Budgeted category name

Column	Type	Constraints	Description
limit_amount	DECIMAL(12,2)	NOT NULL, > 0	Monthly spending cap
month_year	VARCHAR(7)	NOT NULL (YYYY-MM)	Budget month/year period

◆ Table: goals

Column	Type	Constraints	Description
id	UUID	PK	Unique goal identifier
user_id	UUID	FK → users.id	Owning user reference
title	VARCHAR(100)	NOT NULL	Goal name/description
target_amount	DECIMAL(12,2)	NOT NULL, > 0	Savings target value
current_amount	DECIMAL(12,2)	DEFAULT 0.00	Accumulated savings
target_date	DATE	NULLABLE	Target completion date

◆ Table: notifications

Column	Type	Constraints	Description
id	UUID	PK	Unique notification identifier
user_id	UUID	FK → users.id	Target user reference
message	TEXT	NOT NULL	Notification body content
type	VARCHAR(30)	NOT NULL	Category: budget goal report reminder
is_read	BOOLEAN	DEFAULT FALSE	Read status flag
created_at	TIMESTAMPTZ	DEFAULT NOW()	Notification creation time

6. API Design

All APIs follow RESTful conventions with JSON payloads. Every protected endpoint requires a valid Bearer JWT in the Authorization header. Responses use standard HTTP status codes with a consistent error response envelope.

◆ Authentication APIs

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Create a new user account	No
POST	/api/auth/verify-otp	Verify email with OTP code	No
POST	/api/auth/login	Authenticate and receive JWT token	No
POST	/api/auth/forgot-password	Initiate password reset via OTP	No
POST	/api/auth/refresh	Obtain new JWT using refresh token	Yes

◆ Transaction APIs

Method	Endpoint	Description	Auth Required
GET	/api/transactions	Retrieve paginated transactions (filters supported)	Yes
POST	/api/transactions	Create a new transaction record	Yes
PUT	/api/transactions/:id	Update an existing transaction by ID	Yes
DELETE	/api/transactions/:id	Permanently delete a transaction	Yes

◆ Budget APIs

Method	Endpoint	Description	Auth Required
GET	/api/budgets	Retrieve all budgets for current month	Yes
POST	/api/budgets	Create a new category budget	Yes
PUT	/api/budgets/:id	Update an existing budget limit	Yes
DELETE	/api/budgets/:id	Remove a category budget	Yes

◆ Analytics APIs

Method	Endpoint	Description	Auth Required
GET	/api/analytics/summary	Retrieve aggregated financial summary	Yes
GET	/api/analytics/trends	Retrieve spending trend data by period	Yes
GET	/api/analytics/insights	Retrieve rule-generated behavioral insights	Yes

◆ Goals APIs

Method	Endpoint	Description	Auth Required
GET	/api/goals	Retrieve all active savings goals	Yes
POST	/api/goals	Create a new savings goal	Yes
PUT	/api/goals/:id	Update goal amount or deadline	Yes
DELETE	/api/goals/:id	Remove a savings goal	Yes

7. Security Design

Security is a first-class concern at every layer of the FinNewt.bi architecture. The following controls are implemented across the authentication, transport, application, and data layers.

◆ Authentication Flow

1. User submits credentials (POST /api/auth/login)
2. Auth Service validates credentials against BCrypt-hashed DB record
3. On success → JWT (access token, 15min) + Refresh Token (7 days) issued
4. Tokens returned in HTTP-only, Secure, SameSite cookies
5. All subsequent API calls attach Bearer token in Authorization header
6. API Gateway validates JWT signature and expiry on every protected route
7. On token expiry → client uses refresh token to silently re-authenticate

◆ Security Controls Matrix

Control	Implementation	Layer
Password Hashing	BCrypt, cost factor 12	Application
Transport Security	TLS 1.2+, HSTS headers enforced	Transport
Session Management	JWT access (15min) + Refresh (7d) tokens	Application
OTP Verification	SMTP OTP, 10-min expiry, 3 attempts max	Application
Rate Limiting	Redis sliding window: 100 req/min on auth	API Gateway
Input Sanitization	Joi/Bean Validation on all request bodies	Application
SQL Injection	Parameterized queries / JPA prepared statements	Database
XSS Prevention	CSP headers, output encoding in React	Frontend
CORS Policy	Allowlist-based origin validation in API Gateway	API Gateway
Secrets Management	Environment variables, no secrets in codebase	Infrastructure
Audit Logging	All auth events and data mutations logged	Application

8. Insights Engine Logic

The Insights Engine is a rule-based processing module within the Analytics Service. It evaluates aggregated transaction data against a configurable rule set to produce actionable, personalized financial recommendations.

◆ Rule Set — Version 1.0

Rule ID	Trigger Condition	Generated Insight Message
INS-001	Category spend > defined budget limit	'[Category] budget exceeded. Consider adjusting allocations.'
INS-002	Weekend spend > weekday average × 1.5	'Weekend spending is significantly above your weekday average.'
INS-003	Food spend > 40% of total monthly expenses	'Food costs represent a large share of spend. Review for savings.'
INS-004	Savings rate < 10% of monthly income	'Your current savings rate is below the recommended 10%. Consider increasing contributions.'
INS-005	3+ transactions to same merchant in 7 days	'Recurring spend detected at [Merchant]. Check if a subscription or recurring bill.'
INS-006	Month-over-month total expense increase > 20%	'Expenses have risen significantly compared to last month.'
INS-007	Goal progress < expected linear trajectory	'You are behind your savings goal target for [Goal Name].'

◆ Sample Rule Implementation (Pseudo-code)

```
// INS-001: Budget Overspend Detection
for each category in user.budgets:
    totalSpend = sum(transactions where category=cat AND month=current)
    if (totalSpend > budget.limit_amount):
        insight(INS-001, category, totalSpend - budget.limit_amount)

// INS-002: Weekend Spending Pattern
weekendTotal = sum(expenses where dayOfWeek IN [SAT, SUN])
weekdayAvg = sum(expenses where dayOfWeek NOT IN [SAT,SUN]) / 5
if (weekendTotal > weekdayAvg * 1.5):
    insight(INS-002, weekendTotal, weekdayAvg)
```


9. UI/UX Requirements

9.1 Design Language

Attribute	Specification
Visual Style	Modern fintech aesthetic — clean, data-dense, professional
Color Scheme	Deep navy/dark blue primary, electric blue accents, white text
Typography	Inter or Geist font family; defined type scale (xs to 3xl)
Dark / Light Mode	System-preference detection with manual toggle override
Animations	Framer Motion — 150–300ms transitions, spring physics on cards
Component Style	Rounded corners (8px), glassmorphism card surfaces, soft shadows
Iconography	Lucide or Heroicons — consistent 20px stroke icons throughout
Layout Grid	12-column CSS Grid, 4–8 breakpoints (320px to 1920px)
Spacing System	4px base unit, multiples thereof for padding and margins

9.2 User Experience Goals

- Minimal learning curve — new users reach core functionality within 2 minutes
- Fast perceived performance via skeleton screens and optimistic UI updates
- Visual clarity — information hierarchy guides users through data naturally
- Real-time feedback — form validations, save confirmations, and error states inline
- Accessible — keyboard navigable, screen reader compatible, sufficient contrast ratios

10. Future Enhancements

The following capabilities are planned for subsequent release cycles. They are excluded from v1.0 scope but architecturally accommodated in the current design.

Feature	Description	Target Phase
AI Financial Assistant	Conversational LLM-powered advisor via chat interface	Phase 2
OCR Receipt Scanning	Camera-based receipt ingestion auto-creating transactions	Phase 2
Bank Integration	Open Banking API / Plaid integration for auto-import	Phase 3
UPI Sync	Real-time UPI transaction sync (India-specific)	Phase 3
Voice Expense Logging	Speech-to-text quick transaction entry	Phase 2
Multi-User Family Wallet	Shared budget and expense tracking for households	Phase 3
Investment Tracking	Portfolio monitoring with P&L and asset allocation views	Phase 4
Predictive Analytics	ML-based spending forecasting and anomaly detection	Phase 4
Subscription Intelligence	Auto-detect and manage recurring subscriptions	Phase 2

Document Control: This SRS constitutes the authoritative requirements baseline for FinNewt.bi v1.0. All functional and non-functional requirements herein are subject to formal change control. Any deviation must be approved by the Engineering Lead and recorded in the change log. Last reviewed: 2025.